

Chapter 5

Vi: A Text Editor

5.1 Introduction 5-1

5.2 Demonstration 5-1

- 5.2.1 Entering the Editor 5-2
- 5.2.2 Inserting Text 5-3
- 5.2.3 Repeating a Command 5-3
- 5.2.4 Undoing a Command 5-4
- 5.2.5 Moving the Cursor 5-5
- 5.2.6 Deleting 5-6
- 5.2.7 Searching for a Pattern 5-9
- 5.2.8 Searching and Replacing 5-11
- 5.2.9 Leaving Vi 5-13
- 5.2.10 Adding Text From Another File 5-13
- 5.2.11 Leaving Vi Temporarily 5-14
- 5.2.12 Turning on Line Numbering 5-14
- 5.2.13 Exiting the Editor 5-15

5.3 Editing Tasks 5-15

- 5.3.1 Entering the Editor 5-15
- 5.3.2 Moving the Cursor 5-16
- 5.3.3 Moving Around in a File: Scrolling 5-19
- 5.3.4 Inserting Text 5-20
- 5.3.5 Correcting Typing Errors 5-20
- 5.3.6 Opening a New Line 5-20
- 5.3.7 Repeating the Last Insertion 5-21
- 5.3.8 Inserting Text from Other Files 5-21
- 5.3.9 Inserting Control Characters into Text 5-25
- 5.3.10 Joining and Breaking Lines 5-25
- 5.3.11 Deleting a Character: x and X 5-25
- 5.3.12 Deleting a Word: dw 5-25
- 5.3.13 Deleting a Line: D and dd 5-26
- 5.3.14 Deleting an Entire Insertion 5-27

- 5.3.15 Deleting and Replacing Text 5-27
- 5.3.16 Moving Text 5-30
- 5.3.17 Searching: / and ? 5-34
- 5.3.18 Searching and Replacing 5-35
- 5.3.19 Pattern Matching 5-37
- 5.3.20 Undoing a Command: u 5-39
- 5.3.21 Repeating a Command: . 5-39
- 5.3.22 Leaving the Editor 5-39
- 5.3.23 Editing a Series of Files 5-40
- 5.3.24 Editing a New File Without Leaving the Editor 5-42
- 5.3.25 Leaving the Editor Temporarily: Shell Escapes 5-43
- 5.3.26 Performing a Series of Line-Oriented Commands: Q 5-44
- 5.3.27 Finding Out What File You're In 5-44
- 5.3.28 Finding Out What Line You're On 5-45

4 Solving Common Problems 5-45

- 5.5 Setting Up Your Environment 5-46
 - 5.5.1 Setting the Terminal Type 5-47
 - 5.5.2 Setting Options: set 5-47
 - 5.5.3 Displaying Tabs and End-of-Line: list 5-48
 - 5.5.4 Ignoring Case in Search Commands: ignorecase 5-48
 - 5.5.5 Displaying Line Numbers: number 5-49
 - 5.5.6 Printing the Number of Lines Changed: report 5-49
 - 5.5.7 Changing the Terminal Type: term 5-49
 - 5.5.8 Shortening Error Messages: terse 5-49
 - 5.5.9 Turning Off Warnings: warn 5-50
 - 5.5.10 Permitting Special Characters in Searches: nomagic 5-50
 - 5.5.11 Limiting Searches: wrapscan 5-50
 - 5.5.12 Turning on Messages: mesg 5-50
 - 5.5.13 Customizing Your Environment: The .exrc File 5-51

5.6 Command Summary 5-51

- 5.6.1 Entering Vi 5-51
- 5.6.2 Cursor Movement 5-52
- 5.6.3 Inserting Text 5-53
- 5.6.4 Delete Commands 5-53
- 5.6.5 Change Commands 5-53
- 5.6.6 Search Commands 5-54

- 5.6.7 Search and Replace Commands 5-54
- 5.6.8 Pattern Matching: Special Characters 5-54
- 5.6.9 Leaving Vi 5-55
- 5.6.10 Options 5-55



5.1 Introduction

The XENIX system provides two text editors, **ed** and **vi**, that let you create and modify any ASCII text file. This chapter discusses the use of **vi** (Appendix A presents techniques for using **ed**), and is organized as follows:

See Section	For
5.2	A hands-on demonstration of basic vi concepts
5.3	Reference to specific editing tasks
5.4	Problem solving information
5.5	Information on setting up your vi environment
5.6	A summary of vi commands

Because **vi** is such a powerful editor, it has many more commands than you can learn at one sitting. If you have not used a text editor before, the best approach is to become thoroughly comfortable with the concepts and operations presented in the demonstration section, then refer to the second part for specific tasks you need to perform. Once you are familiar with the basic **vi** commands, you can easily learn to use the more advanced features.

If you have used a text editor before, you may want to turn directly to the task-oriented part of this chapter. Begin by learning the features you will use most often.

This chapter covers the basic text editing features of **vi**. For more advanced topics, and features related to editing programs, refer to **vi(C)** in the XENIX *Reference Manual*.

5.2 Demonstration

The following demonstration gives you hands-on experience using **vi**, and introduces some basic concepts that you must understand before you can learn more advanced features. This demonstration should take one hour. Remember that the best way to learn **vi** is to actually use it, so don't be afraid to experiment.

Before you start the demonstration, make sure that your terminal has been properly set up. See Section 5.5.1 for more information about setting up your terminal for use with **vi**.

Note

Throughout this chapter, the character, word or line affected by the cursor is referred to as being *current*: e.g., the *current line* is the line on which the cursor resides.

5.2.1 Entering the Editor

To enter the editor and create a file named *temp*, type

vi temp

Your screen will look like this:

```
~
~
~
~
~
~
~
~
~
~
~
~
~
"temp" [New file]
```

Note that we show a twelve-line screen to save space. In reality, **vi** uses whatever size screen you have.

The top line of your display is the only line in the file and is marked by the cursor, shown as an underline character; tildes indicate lines on the screen only, not real lines in the file.

5.2.2 Inserting Text

To begin, create some text in the file *temp* by using the **i** (insert) command. To do this, press

i

Like most **vi** commands, the **i** command doesn't appear on your screen. The command itself switches you from *command mode* to *insert mode*:

- In *insert mode* every character you type appears on the screen.
- In *command mode* XENIX interprets the characters you type as commands: the characters don't appear on screen.

If you are not certain which mode you are in, press ESC until you hear the bell: this indicates that you are in command mode.

Next, type the following five lines. Press RETURN at the end of each line. If you make a mistake, use the BKSP key to erase the error.

```
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
```

```
=
~
~
~
~
~
```

Press ESCAPE when you are finished.

5.2.3 Repeating a Command

The repeat command repeats the most recent insertion or deletion. To execute the repeat command, press the period (.).

So, to add five more lines of text, enter command mode, and press “.”; your screen will look like this:

```
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.
```

~

The repeat command is repeated relative to the location of the cursor and inserts text *below* the current line.

5.2.4 Undoing a Command

Another useful command is the **undo** command, **u**. Undo lets you recover from inadvertent deletions or insertions. Press

u

and notice that the five lines you just finished inserting are deleted or “undone.”

```
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.
```

~

~

~

~

~

~

Now type

u

again, and the five lines are reinserted.

5.2.5 Moving the Cursor

In addition to the arrow keys, the following letter keys let you move the cursor around the screen. Since you're still in command mode, give them a try.

Press	To move the cursor
-------	--------------------

h	Left
l	Right
k	Up
j	Down

Remember that the cursor movement keys work only in command mode. You may want to try the following additional cursor movement commands:

Press	To move the cursor
-------	--------------------

w	Forward one word.
b	Back one word.
0	To the beginning of a line.
\$	To the end of a line.
(n) G	To the specified line in the file. G alone places the cursor on the last line in the file.
H	To the upper-left corner of the screen.
M	To the middle line on the screen.
L	To the bottom line on the screen.

You can move through many lines quickly with the scrolling commands:

Press	To scroll
-------	-----------

CTRL-U	Up 1/2 screen
CTRL-D	Down 1/2 screen
CTRL-F	Forward one screenful

CTRL-B Back one screenful

5.2.6 Deleting

Now that we know how to insert and create text, and how to move around within the file, we're ready to delete text. Many delete commands can be combined with cursor movement commands, as explained below. The most common delete commands are

dd	Deletes the current line, regardless of the location of the cursor in the line.
dw	Deletes the current word. If the cursor is in the middle of the word, deletes from the cursor to the end of the word.
x	Deletes the current character.
d\$	Deletes from the cursor to the end of the line.
D	Deletes from the cursor to the end of the line.
d0	Deletes from the cursor to the beginning of the line.

To learn how these commands work, we'll delete various parts of the demonstration file.

1. Press ESC to enter command mode.
2. Move to the first line of the file by pressing

1G

At first, your file should look like this:

```
[F]iles contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
~
~
```

Note

Throughout this chapter the current character will be bracketed ([]).

3. Delete the current line by typing

dd

Your file should now look like this:

```
[T]ext contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~
```

4. Delete the current word by typing

dw

Your file should look like this:

```
[c]ontains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~
```

5. Delete the current character above the cursor by pressing

x

Your file should look like this:

```
[o]ntains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~
```

6. Type a “w” command to move your cursor to the beginning of the word “lines” on the first line. Delete to the end of the line by typing

d\$

Your file looks like this:

```
ontains_  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~
```

7. Delete all the characters on the line *before* the cursor by typing

d0

This leaves a single space on the line:

```

Lines contain characters.
Files contain text.
Text contains lines.
Characters form words.
Words form text.
Lines contain characters.
Characters form words.
Words form text.
~
~
~

```

For review, let's restore the first two lines of the file.

8. Press "i" to enter insert mode, then type

```

Files contain text.
Text contains lines.

```

9. Press ESC to return to command mode.

5.2.7 Searching for a Pattern

You can search forward in a file for a pattern of characters by typing a slash (/) followed by the pattern you are searching for, terminated by a RETURN. For example, make sure you are in command mode (press ESC), then press

H

to move the cursor to the top of the screen. Now, type

/char

Don't press RETURN yet. Your screen should look like this:

```
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
_
~
/char_
```

Press RETURN. The cursor moves to the beginning of the word "characters" on line three. To search for the next occurrence of the pattern "char", press "n" (as in "next"). This will take you to the beginning of the word "characters" on the eighth line. (The case-sensitive system will bypass the capitalized "Characters" on line four.) If you keep pressing "n" vi searches past the end of the file, wraps around to the beginning, and again finds the "char" on line three.

Note that the slash character and the pattern that you are searching for appear at the bottom of the screen. This bottom line is the *vi status line*. The status line displays information, including patterns you are searching for, line-oriented commands (explained later in this demonstration), and error messages.

For example, to get status information about the file, press CTRL-G. Your screen should look like this:

```
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
Files contain text.
Text contains lines.
Lines contain [c]haracters.
Characters form words.
Words form text.
_
"temp" [Modified] line 4 of 10 --40%--
```

The status line tells you

- the name of the file
- whether it has been modified
- the current line number
- the number of lines in the file
- your location in the file as a percentage of the number of lines in the file

The status line disappears as you continue working.

5.2.8 Searching and Replacing

The XENIX search and replace feature lets you use one command to change all occurrences of a text string. The command will appear on the status line as you type it. For example, to change “text” to “documents”:

1. Press ESC to enter command mode.
2. Type

```
:1,$s/text/documents/g
```

This command means “From the first line (1) to the end of the file (\$), find *text* and replace it with *documents* (s/text/documents/) everywhere it occurs on each line (g)”.

Press RETURN. Your screen should look like this:

```
Files contain documents.
Text contains lines.
Lines contain characters.
Characters form words.
Words form documents.
Files contain documents.
Text contains lines.
Lines contain characters.
Characters form words.
[W]ords form documents.
_
-
```

Note that "Text" in lines two and eight was not changed. Case is significant in searches.

Just for practice, use the undo command to change "documents" back to "text". Press

u

Your screen now looks like this:

```
[F]iles contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~
```

Most of the commands you have learned so far have been *screen-oriented*: screen-oriented commands execute at the location of the cursor. You do not need to tell the computer where to perform the operation; it takes place relative to the cursor. Commands that can perform more than one action (searching and replacing) are *line-oriented* commands: line-oriented commands require you to specify an exact location (called an "address") where the operation is to take place. Screen-oriented commands are easy to type in, and provide immediate feedback; the change is displayed on the screen. Line-oriented commands are more complicated to type in, but they can be executed independent of the cursor, and in more than one place in a file at a time.

All line-oriented commands are preceded by a colon which acts as a prompt on the status line. Line-oriented commands themselves are entered on this line and terminated with a RETURN.

Note

In this chapter, all instructions for line-oriented commands will include the colon as part of the command.

5.2.9 Leaving Vi

All of your editing has affected a *copy* of the file *temp*, that you specified when you invoked **vi**. To save these changes, exit the editor and return to the XENIX shell by typing

```
:x
```

The name of the file and the number of lines and characters it contains appear on the status line:

```
"temp" [New file] 10 lines, 214 characters
```

The XENIX prompt will then appear below the status line.

5.2.10 Adding Text From Another File

In this section we'll create a new file, and insert text from another file into it.

1. Create a new file named *practice*:

```
vi practice
```

2. Press **ESC** to enter command mode.
3. Use the line-oriented **read** command to copy the text from *temp* to *practice*:

```
:r temp
```

Your file should look like this:

```
[F]iles contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
_
```

There is an empty line at the top of the file. Move the cursor to the empty line and delete it with the **dd** command.

5.2.11 Leaving Vi Temporarily

You can exit **vi** temporarily to execute commands by preceding the command with an exclamation point on the status line. For example, to find out the date and time, type

```

:!date

```

This displays the date, then prompts you to press RETURN to reenter command mode. Your screen should look similar to this:

```
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
_
: !date
Mon Jan 9 16:33:37 PST 1984
[Hit return to continue]
```

5.2.12 Turning on Line Numbering

Vi lets you control a number of editing parameters such as line number display.

To turn on automatic line numbering, type

```
:set number
```

XENIX redraws your screen, and places line numbers to the left of the text:

```

1 Files contain text.
2 Text contains lines.
3 Lines contain characters.
4 Characters form words.
5 Words form text.
6 Files contain text.
7 Text contains lines.
8 Lines contain characters.
9 Characters form words.
10 Words form text.
-
-
```

To see the complete list of available options, type

```
:set all
```

See Section 5.5 for information on setting these options.

5.2.13 Exiting the Editor

To exit **vi** without saving the changes made to the file, type

```
:q!
```

This completes the demonstration. You have learned the fundamentals of **vi**. The following sections provide more detailed information about these fundamentals and about **vi**'s other commands and features.

5.3 Editing Tasks

Sections 5.3.1 through 5.3.28 explain how to perform common editing tasks.

5.3.1 Entering the Editor

There are several ways to begin editing, depending on what you are planning to do.

5.3.1.1 Entering Vi with a Filename

The most common way to enter **vi** is to type “**vi**” and the name of the file you wish to edit:

```
vi filename
```

If *filename* does not already exist, a new, empty file is created.

5.3.1.2 Entering at a Particular Line

You can also enter the editor at a particular place in a file. For example, to start editing a file at line 100, type

```
vi +100 filename
```

The cursor is placed at line 100 of *filename*.

5.3.1.3 Entering at a Particular Word

To begin editing at the first occurrence of a particular word, type

```
vi +/word filename
```

The cursor is placed at the first occurrence of *word*.

Section 5.3.23 describes how to invoke **vi** on more than one file at a time.

5.3.2 Moving the Cursor

Cursor movement keys let you move the cursor within a file. You must be in command mode to move the cursor.

5.3.2.1 Moving the Cursor By Character

Press	To move the cursor
SPACEBAR or l	forward
BACKSPACE or h	backward
k	up
j	down

The cursor moves one character, unless you specify a number of characters. For example, to move backward four characters, type

```
4h
```

To move the cursor to a designated character on the current line, use the “f” key:

Press	To move the cursor
F	backward
f	forward

For example, to move the cursor backward to the nearest *p* on the current line, type

Fp

To move the cursor forward to the nearest *p*, type

fp

The T and t keys work very similar to the f and F keys. The T key places the cursor immediately *after* the preceding occurrence of the character, the t key places the cursor immediately *before* the following occurrence of the character. For example, if the cursor was resting on the r in the word “whatever,” the following command would move the cursor to the “a”:

Th

The cursor always remains on the same line when you use these commands. If you specify a character not on the line, the command does nothing.

5.3.2.2 Moving the Cursor by Words

The “w” key moves the cursor forward to the *beginning* of the specified number of words. Punctuation and nonalphabetic characters are considered words, and will be counted as such. For example, if the cursor rests on the first letter of the sentence

No, I didn’t know he had returned.

and you press

6w

the cursor stops on the *k* in *know*. The comma and apostrophe are considered words.

W works like w, but includes punctuation and nonalphabetic characters as part of the word. Using the previous example, if you press

6W

the cursor stops on the *r* in *returned*; the comma and the apostrophe are considered part of the words they punctuate.

The e and E keys move the cursor forward to the *end* of a specified number of words. The cursor is placed on the last letter of the word. The e command counts punctuation and nonalphabetic characters as separate words; E does not.

B and b move the cursor back to the beginning of a specified number of words. The cursor is placed on the first letter of the word. The b command counts punctuation and nonalphabetic characters as separate words; B does not. Using the previous example, if the cursor is on the *r* in *returned*, type

4b

and the cursor moves to the *t* in *didn't*. Type

4B

and the cursor moves to the first *d* in *didn't*.

The w,W,b and B commands will move the cursor to the next line if that is where the designated word is, unless the current line ends in a space.

5.3.2.3 Moving the Cursor by Lines

The RETURN, LINEFEED and “+” keys move the cursor forward a specified number of lines, placing the cursor on the first character. For example, to move the cursor forward six lines, type

6+

The “j” and CTRL-N keys move the cursor forward a specified number of lines. The cursor remains in the same place on the line, unless there is no character in that place, in which case it moves to the last character on the line. For example, in the following two lines, if the cursor is resting on the *e* in “characters”, pressing “j” moves it to the period at the end of the second line:

Lines contain characters.
Text contains lines.

The dollar sign(\$) moves the cursor to the end of a specified number of lines. For example, to move the cursor to the last character of the line four lines down from the current line, type

4\$

CTRL-P and k move the cursor backward a specified number of lines, keeping

it on the same place on the line. For example, to move the cursor backward four lines from the current line, type

4k

5.3.2.4 Moving to Main Screen Locations

The H, M and L keys move the cursor to the beginning of the top, middle and bottom lines of the screen, respectively.

5.3.3 Moving Around in a File: Scrolling

The following commands let you display different portions of the file on screen:

Press	To scroll
CTRL-U	Up one-half screen
CTRL-B	Up one full screen
CTRL-D	Down one-half screen
CTRL-F	Down one full screen

5.3.3.1 Placing a Line at the Top of the Screen

To scroll the current line to the top of the screen, type

z

and press RETURN. To place a specific line at the top of the screen, precede “z” with the line number, as in

33z

For information on how to display line numbers, see Section 5.5.5.

5.3.4 Inserting Text

You can insert text anywhere on a line. In order to insert text into a file, you must be in *insert mode*. To enter insert mode, press one of the following keys:

Press	To enter insert mode
-------	----------------------

i	At the current cursor location (text inserted <i>before</i> the cursor)
I	At the beginning of the current line
a	At the current cursor location (text inserted <i>after</i> the cursor)
A	At the end of the current line

The characters don't appear on the screen, but any text typed after them becomes part of the file you are editing.

To leave insert mode and reenter command mode, press ESC. For more explanation of modes in vi, see Section 5.2.2.

5.3.5 Correcting Typing Errors

To correct a typing error on the current line, backspace over the mistake then retype the line. See Sections 5.3.11 through 5.3.15 for other ways to correct typing mistakes.

5.3.6 Opening a New Line

You can use several methods to open new lines in a file:

Press	To open a new line
-------	--------------------

O	Above the current line
o	Below the current line

Both commands place you in insert mode; you may enter text immediately.

If you're in insert mode, you can also use the RETURN key to open new lines. To open a line above the current line, move the cursor to the beginning of the line, enter insert mode, then press RETURN. (For information on how to move the cursor, see Section 5.3.2.) To open a line below the current line, move the cursor to the end of the line, enter insert mode, then press RETURN.

5.3.7 Repeating the Last Insertion

CTRL-@ repeats the last insertion. Enter insert mode, then press CTRL-@.

CTRL-@ repeats insertions of 128 characters or less. If more than 128 characters were inserted, CTRL-@ does nothing.

For other methods of repeating an insertion, see Sections 5.3.8, Section 5.3.9, and Section 5.3.16.

5.3.8 Inserting Text from Other Files

To insert the contents of another file into the current file, use the **read** command.

1. Move the cursor to the line immediately *above* the intended location of the new material.
2. Type

:r filename

where *filename* is the file containing the material to be inserted, and press RETURN.

The text of *filename* appears on the line below the cursor, and the cursor moves to the first character of the new text. This text is a copy; the original *filename* still exists.

To insert selected lines from another file, you first copy them from the original file into a temporary holding place called a *buffer*, and then insert them into the new file.

1. Use the **write** command (**:w**) to save the file you're copying to. Do not exit vi.
2. Type

:e filename

where *filename* is the file that contains the text you want to copy. Press RETURN.

3. When the file appears on screen, move the cursor to the first line you wish to select.
4. Mark this line by typing

mk

5. Move the cursor to the last line of the text you wish to select. Type

"ay'k

This command “yanks” the lines from marked location to the cursor, and places them into buffer *a*. They will remain in buffer *a* until you replace them with other lines, or until you exit the editor.

6. Type

:e#

to return to your previous file. (For more information about this command, see Section 5.3.25.)

7. Move the cursor to the line above the intended location of the new text, and type

"ap

This puts a copy of the yanked lines from buffer *a* into the file, and places the cursor on the first letter of this new text. The buffer still contains the original yanked lines.

You can have 26 buffers named *a*, *b*, *c*, up to and including *z*. To name and select different buffers, replace the *a* in steps 5 and 7 with whatever letter you wish.

You may also delete text into a buffer, then insert it in another place. For information on this type of deletion and insertion, see Section 5.3.16.

5.3.8.1 Copying Lines from Elsewhere in the File

To copy lines from one place in a file to another, use the **copy(co)** command.

Co is a line-oriented command, and to use it you must know the line numbers of the text to be copied and its destination. To find out the number of the current line, type

:nu

and press RETURN. The line number and the text of that line appear on the status line. To find out the destination line number, move the cursor to the line above where you want the copied text to appear and repeat the **:nu** command. You can also make line numbers appear throughout the file with the **number** option. For information on how to set this option, see Section 5.5.5. The fol-

Following example uses the **number** option.

```

1 [F]iles contain text.
2 Text contains lines.
3 Lines contain characters.
4 Characters form words.
5 Words form text.
~
~
~
~
~
~

```

Using the above example, to place copies of lines 3 and 4 between lines 1 and 2, type

```
:3,4 co 1
```

The result is

```

1 Files contain text.
2 Lines contain characters.
3 [C]haracters form words.
4 Text contains lines.
5 Lines contain characters.
6 Characters form words.
7 Words form text.
~
~
~
~
~

```

To insert text several times in different places, you can save it in a buffer, and insert it whenever it is needed. For example, to repeat the first line of the following text after the last line:

```
[F]iles contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
_  
_  
_  
_  
_  
_  
_  
_
```

1. Move the cursor over the "F" in "Files." Type the following line, which will not appear on your screen:

"ayy

This "yanks" a copy of the first line into buffer *a*. Move the cursor over the "W" in "Words."

2. Type the following line:

"ap

This puts a copy of the yanked line into the file, and places the cursor on the first letter of this new text. The buffer still contains the original yanked line.

Your screen looks like this:

```
Files contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
[F]iles contain text.  
_  
_  
_  
_  
_  
_  
_
```

To yank several consecutive lines, indicate the number of lines you wish to yank after the name of the buffer. For example, to place three lines from the above text in the buffer *a*, type

"a3yy

For another method of placing text in a buffer, and more information about naming buffers, see Section 5.3.8.

5.3.9 Inserting Control Characters into Text

Many control characters have special meaning in **vi**, even when typed in insert mode. To remove their special significance, press CTRL-V before typing the control character. Note that CTRL-J, CTRL-Q, and CTRL-S cannot be inserted as text: CTRL-J is a newline character; CTRL-Q and CTRL-S are meaningful to the operating system, and are trapped by it before they are interpreted by **vi**.

5.3.10 Joining and Breaking Lines

To join two lines, press

J

while the cursor is on the first of the two lines you wish to join.

To break one line into two lines, position the cursor on the space preceding the first letter of what will be the second line, press

r

then press RETURN.

5.3.11 Deleting a Character: x and X

The **x** and **X** commands delete a specified number of characters. The **x** command deletes the current character; the **X** command deletes the character immediately before the cursor. If no number is specified, one character is deleted. For example, to delete three characters following the cursor (including the current character), type

3x

To delete three characters preceding the cursor, type

3X

5.3.12 Deleting a Word: dw

The **dw** command deletes a specified number of words. If no number is given, one word is deleted. A word is interpreted as numbers and letters separated by white space. When a word is deleted, the space after it is also deleted. For ex-

ample, to delete three words, type

3dw

5.3.13 Deleting a Line: D and dd

The **D** command deletes all text following the cursor on that line, including the current character. The **dd** command deletes a specified number of lines and closes up the space. If no number is given, only the current line is deleted. For example, to delete three lines, type

3dd

Another way to delete several lines is to use a line-oriented command. To use this command it helps to know the line numbers of the text you wish to delete. For information on how to display line numbers, see Section 5.5.5.

For example, to delete lines 200 through 250, type

:200,250d

Press RETJRN. When the command finishes, the message

50 lines

appears on the **vi** status line, indicating the number of lines deleted.

It is possible to remove lines without displaying line numbers using shorthand "addresses." For example, to remove all lines from the current line to the end of the file, type

:\$d

The dot (.) represents the current line, and the dollar sign stands for the last line in the file. To delete the current line and 3 lines following it, type

.,+3d

To delete the current line and 3 lines preceding it, type

.,-3d

For more information on using addresses in line-oriented commands, see *vi(C)* in the *XENIX Reference Manual*.

5.3.14 Deleting an Entire Insertion

To delete all of the text you just typed, press CTRL-U while you are in insert mode. The cursor returns to the beginning of the insertion. The text of the original insertion is still displayed, and any text you type replaces it. When you press ESC, any text remaining from the original insertion disappears.

5.3.15 Deleting and Replacing Text

Several vi commands combine removing characters and entering insert mode. The following sections explain how to use these commands.

5.3.15.1 Overstriking: r and R

The **r** command replaces the current character with the next character typed. To replace the current character with a “b”, for example, type

```
rb
```

If a number precedes **r**, that number of characters are replaced with the next character typed. For example, to replace the current character, plus the next three characters, with the letter “b”, type

```
4rb
```

Note that you now have four “b”s in a row.

The **R** lets you overstrike characters up to the end of the current line. To end the replacement, press ESC. For example, to replace the second line in the following text with “Spelling is important.”:

```
Files contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
~
~
~
~
~
~
```

1. Move the cursor over the “T” in “Text.”

2. Press **R**, then type

Spelling is important.

3. Press **ESC** to end the replacement. If you make a mistake, use the **BKSP** key to correct it. Your screen should now look like this:

```
Files contain text.  
Spelling is important[.]  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~  
~  
~  
~  
~
```

5.3.15.2 Substituting: **s** and **S**

The **s** command replaces a specified number of characters with text you type in, beginning with the current character. The **s** command leaves you in insert mode, so you must press **ESCAPE** to exit insert mode.

To substitute *xyz* for the character under the cursor and the two characters following it, type

```
3sxyz
```

The **S** command deletes a specified number of lines and replaces them with text you type in. You may type in as many new lines of text as you wish; press **ESCAPE** to exit the command. The **S** command affects only how many lines are deleted. If no number is provided, one line is deleted. For example, to delete four lines, including the current line, type

```
4S
```

This differs from the **R** command. The **S** command deletes the entire current line; the **R** command deletes text from the cursor onward.

5.3.15.3 Replacing a Word: **cw**

The **cw** command replaces a word with text you type in. For example, to replace the word "bear" with the word "fox," move the cursor over the "b" in

“bear.” Type

cw

A dollar sign appears after the “r” in “bear,” marking the end of the text that is being replaced. Type

fox

and press ESCAPE. The rest of “bear” disappears and only “fox” remains.

5.3.15.4 Replacing the Rest of a Line: C

The **C** command replaces text from the cursor to the end of the line. For example, to replace the text of the sentence

Who’s afraid of the big bad wolf?

from “big” to the end, move the cursor over the “b” in “big” and press

C

A dollar sign (\$) replaces the question mark (?) at the end of the line. Type the following:

little lamb?

Press ESC. The remaining text from the original sentence disappears.

5.3.15.5 Replacing a Whole Line: cc

The **cc** command deletes a specified number of lines, regardless of the location of the cursor, and replaces them with text you type, until you press ESCAPE. If no number is given, the current line is deleted.

5.3.15.6 Replacing a Particular Word on a Line

If a word occurs several times on one line, it is often convenient to use a line-oriented command to replace it. For example, to replace the word “removing” with “deleting” in the following sentence:

In vi, removing a line is as easy as removing a letter.

Make sure the cursor is at the beginning of that line, and type

:s/removing/deleting/g

Press RETURN. This line-oriented command means “Substitute (s) for the word *removing* the word *deleting*, everywhere it occurs on the current line (g)”. If you don’t include a g at the end, only the first occurrence of *removing* is changed.

For more information on using line-oriented commands to replace text, see Section 5.3.19.

5.3.16 Moving Text

To move a block of text from one place in a file to another, you can use the line-oriented **m** command. You must know the line numbers of your file to use this command. The **number** option displays line numbers. To set this option, enter command mode, then type

set number

Line numbers will appear to the left of your text. (For more information on setting the **number** option, see Section 5.5.5.

The following example uses the **number** option. For other ways to display line numbers, see Section 5.3.28.

```
1 [F]iles contain text.  
2 Text contains lines.  
3 Lines contain characters.  
4 Characters form words.  
5 Words form text.  
-  
-  
-  
-  
-  
-
```

To move lines 2 and 3 between lines 4 and 5, type

:2,3m4

Your screen should look like this:

```
1 Files contain text.  
2 Characters form words.  
3 Text contains lines.  
4 Lines contain characters.  
5 [W]ords form text.  
~  
~  
~  
~  
~  
~
```

To move line 5 after line 2, type

:5m2

After moving, your screen should look like this:

```
1 Files contain text.  
2 Characters form words.  
3 [W]ords form text.  
4 Text contains lines.  
5 Lines contain characters.  
~  
~  
~  
~  
~  
~
```

To move line 4 to the first line in the file, type

:4m0

Your screen should look like this:

```
1 [T]ext contains lines.
2 Files contain text.
3 Characters form words.
4 Words form text.
5 Lines contain characters.
~
~
~
~
~
~
~
```

You can also delete text into a temporary storage place, called a *buffer*, and insert it wherever you wish. When text is deleted it is placed in a “delete buffer.” There are nine delete buffers.

The first buffer always contains the most recent deletion. In other words, the first deletion in a given editing session goes into buffer 1. The second deletion also goes into buffer 1, and pushes the contents of the old buffer 1 into buffer 2. The third deletion goes into buffer 1, pushing the contents of buffer 2 into buffer 3, and the contents of buffer 1 into buffer 2. When buffer 9 has been used, the next deletion pushes the current text of buffer 9 off the stack and it disappears.

Text remains in the delete buffers until it is pushed off the stack, or until you quit the editor, so it is possible to delete text from one file, change files without leaving the editor, and place the deleted text in another file.

Delete buffers are particularly useful when you wish to remove text, store it, and put it somewhere else. Using the following text as an example

```
[F]iles contain text.
Text contains lines.
Lines contain characters.
Characters form words.
Words form text.
~
~
~
~
~
~
~
```

This means everyone may search and read the directory but only the user may create and remove files and directories within it.

4.3.2 Changing Permissions

You can change the permissions of a file or a directory with the **chmod** (for “change mode”) command. This command requires that you tell it how to change the permissions of a specific file or directory. You do so by indicating which levels of permissions you wish to change (user “u”, group “g”, or other “o”), how you wish to change them (add “+” or remove “-”), and which permissions you wish to change (read “r”, write “w”, or execute “x”). For example, the pattern:

```
u+x
```

adds execute permission for the user. The pattern:

```
go-w
```

removes write permission for group and other.

The **chmod** command has the form:

```
chmod pattern file ...
```

where *pattern* is the new level of permissions and *file* is the name of a file or directory. If more than one name is given, the names must be separated by spaces. For example, to change the permissions of the file “receivables” from “-rw-r--r--” to “-rw-----”, type:

```
chmod go-r receivables
```

Press the RETURN key.

After using **chmod**, use the **ls** command to check the results. If you have made a mistake, use **chmod** again to correct the mistake.

4.3.3 Changing the File Creation Mask

The file creation mask is a special number, kept by the system, that defines the permissions given to every file and directory created by a user. Initially, the mask has the value “022” which means every file receives the permissions:

```
-rw-r--r--
```

Every directory receives the permissions:

```
drwxr-xr-x
```

You can change the mask and the initial permissions your files and directories receive by using the **umask** command.

The **umask** command has the form:

umask *value*

where *value* is a three-digit number. The three digits represent user, group, and other permissions, respectively. The value of a digit defines which permission is given, as shown in the following list:

Digit	Permission
0	Read and write (also execute for directories)
1	Read and write
2	Read (also execute for directories)
3	Read
4	Write (also execute for directories)
5	Write
6	Execute for directories
7	No permissions

For example, the command:

umask 177

sets the file creation mask so that all files and directories initially have read and write permission for the user and no permissions for all others.

4.4 Managing File Ownership

Whenever a file is created by a user, the system automatically assigns *user ownership* of that file to that user. This allows the creator to access the file according to the user permissions. The system also assigns a *group ownership* to the file. The group ownership defines which group may access the file according to the group permissions. The group is the same group to which the user who created the file belongs.

Only one user and one group may have ownership of a file at any one time. (These are the owner and group displayed by the **ls** command.) However, you may change the ownership of a file by using the **chown** and **chgrp** commands.

4.4.1 Changing User Ownership

You can change the user ownership of a file with the **chown** command. The command has the form:

```
chown login-name file ...
```

where *login-name* is the name of the new user and *file* is the name of the file or directory to be changed. For example, the command:

```
chown johnd projects.june
```

changes the current owner of the file *projects.june* to “johnd”.

The **chown** command is especially useful after you change the user ID of a user account (see Section 3.7 in Chapter 3, “Preparing XENIX for Users”).

You must be logged in as the super-user to use this command.

4.4.2 Changing Group Ownership

You can change the group ownership of a file with the **chgrp** command. The command has the form:

```
chgrp group-name file ...
```

where *group-name* is the name of a group given in the */etc/group* file and *file* is the name of the file you wish to change. For example, the command:

```
chgrp shipping projects.june
```

changes the group ownership of the file *projects.june* to the group named “shipping”.

The **chgrp** command is especially useful if you have changed the login group of a user (see Section 3.6 in Chapter 3, “Preparing XENIX for Users”).

4.5 System Security

Every system, no matter what its size, should have some form of protection from unauthorized access to the computer, disks, and system files. The following sections suggest ways for a system manager to protect the system.

4.5.1 Physical Security

You can protect the physical components of the computer, especially system disks, by taking these steps:

1. Keep unessential personnel out of the work area.
2. Organize and lock up all floppy disks when not in use. They should not be stored with the computer itself.
3. Keep disks away from magnetism, direct sunlight, and severe changes in temperature.
4. Do not use ball point pens to write labels on disks.
5. Make backup copies of all floppy disks (see Section 4.5.4 in this chapter).

4.5.2 Access Security

You can protect the system from access by unauthorized individuals by taking these steps:

1. Remind users to log out of their accounts before leaving the terminal.
2. Discourage users from choosing passwords that are easy to guess. Passwords should be at least six characters long and include letters, digits, and punctuation marks.
3. Keep the super-user password secret from all but necessary personnel.

4.5.3 Protecting Special Files

You can prevent ordinary users from gaining direct access to the data and program files on the system's hard and floppy disks by protecting the system's special files. The XENIX special files, in the */dev* directory, are used primarily by the system to transfer data to and from the computer's hard and floppy disks and other devices, but can also be used by ordinary users to gain direct access to these devices.

Since direct access bypasses the system's normal protection mechanisms and allows ordinary users to examine and change all files in the system, it is wise to protect the special files to ensure system security.

To protect the XENIX special files, log in as the super-user and use the **chmod** command to set appropriate permissions. For example, to disallow any access by ordinary users, set the permissions of such special files as */dev/mem*, */dev/kmem*, */dev/root*, and */dev/usr* to read and write access for the user only. Note that you must not change the permissions for the */dev/tty* files.

4.5.4 Copying Floppy Disks

To ensure against the loss of data stored on floppy disks, you can use the **dd** command to make copies of floppy disks on new, formatted disks.

To make a copy of a disk, follow these steps:

1. Insert the disk to be copied into floppy drive 0.
2. Insert an empty, formatted disk into drive 1. If necessary, you can format a disk with the **format** command described in Section 4.2.4 in this chapter.
3. Type:

```
dd if=/dev/fd0 of=/dev/fd1 count=blkcount
```

Press the RETURN key. The *blkcount* must be the number of blocks on the disk you are copying (see Appendix A, “XENIX Special Device Files” for details).

The command copies the first disk to the second, then displays a record of the number of blocks copied.

4.6 Using XENIX Accounting Features

The XENIX system provides a set of commands that allow the system manager to perform process accounting. Process accounting is a simple way to keep track of the amount of time each user spends on the system. The process accounting commands keep a record of the number of processes (i.e., programs) started by a user, how long each process lasts, and other information such as how often the process accesses I/O devices and how big the process is in bytes.

Process accounting is helpful on systems where users are being charged for their access time, and it may also be used to develop a detailed record of system, command, and system resource usage.

There are several commands that may be used to do process accounting. Of these, the most useful are **accton** and **acctcom**. The **accton** command starts and stops process accounting. When invoked, the command copies pertinent information about each process to the file named */usr/adm/pacct*. You may use the **acctcom** command to display this information. The command has several options for displaying different types of accounting information.

4.6.1 Starting Process Accounting

You can start process accounting at any time, but it is typically started when the system itself is started. You start process accounting with the **accton** com-

mand. Type:

```
accton /usr/adm/pacct
```

The command automatically creates a new file */usr/adm/pacct* and begins to copy process accounting information to it. If the */usr/adm/pacct* file exists before starting **accton**, the file contents are deleted.

Note that when you start the system, the contents of the */usr/adm/pacct* file is usually saved in the file */usr/adm/opacct*.

4.6.2 Displaying Accounting Information

The **acctcom** command reads processing accounting information from the */usr/adm/pacct* file by default, then displays selected information on your terminal screen. The command usually displays basic accounting information, such as the process's program name, the name of the user who invoked the process, the start and stop times for the process, and the number of execution seconds in real time and CPU time. The command has several options you can select to display other information.

To display the average size of each process, type:

```
acctcom
```

The command displays the basic information plus the average size of each process.

To display basic accounting information about a specific command, type:

```
acctcom -n command
```

where *command* is the name of the command you are interested in. The command responds by displaying each entry for the specified *command*. For example, to display each entry for the system command **units**, type:

```
acctcom -n units
```

To display information about the number and size of input and output counts, type:

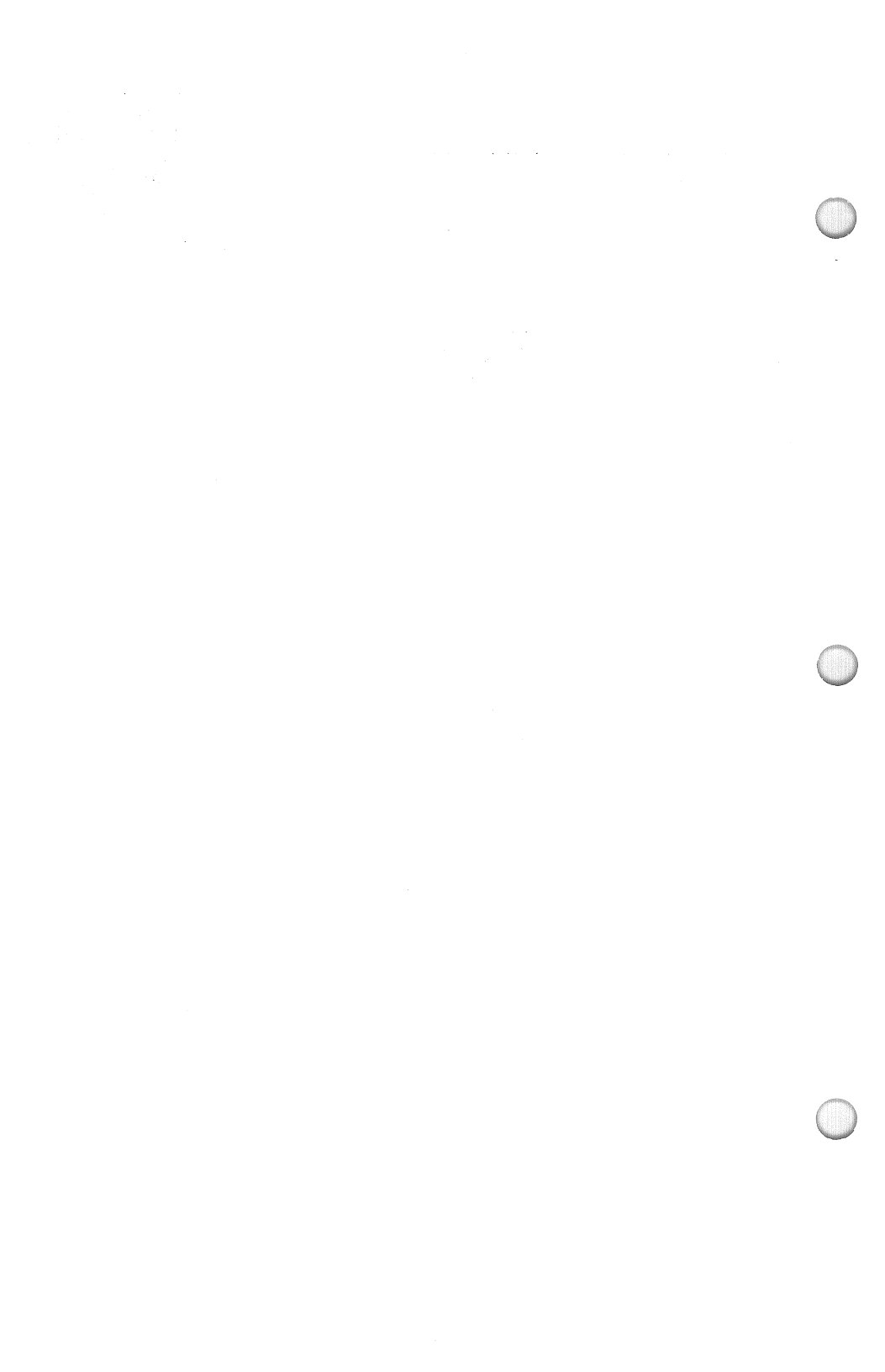
```
acctcom -i
```

The command displays basic program information plus the number of characters and blocks transferred or read by each program.

To display information about a program's use of system resources, type:

acctcom -h

The command displays the basic information plus the *use factor*. The use factor is a number generated and used by the system to determine how each process should be scheduled for execution. Processes with high use factors use a high percentage of the system resources and are therefore scheduled after processes with lower factors.



Chapter 5

Maintaining File Systems

- 5.1 Introduction 5-1**
- 5.2 Maintaining Free Space 5-1**
 - 5.2.1 Strategies for Maintaining Free Space 5-1**
 - 5.2.2 Displaying Free Space 5-2**
 - 5.2.3 Sending a System-Wide Message 5-2**
 - 5.2.4 Displaying Disk Usage 5-3**
 - 5.2.5 Displaying Blocks by Owner 5-3**
 - 5.2.6 Mailing a Message to a User 5-3**
 - 5.2.7 Locating Files 5-4**
 - 5.2.8 Locating core and /tmp Files 5-5**
 - 5.2.9 Clearing Log Files 5-5**
 - 5.2.10 Expanding the File System 5-6**
- 5.3 File System Integrity 5-6**
 - 5.3.1 Repairing the File System 5-6**
 - 5.3.2 Automatic File System Check 5-7**



5.1 Introduction

File system maintenance, an important task of the system manager, keeps the XENIX system running smoothly, keeps the file systems clean, and ensures adequate space for all users. To maintain the file systems, the system manager must monitor the free space in each file system, and take corrective action whenever it gets too low.

This chapter explains the file system maintenance commands. These commands report how much space is used, locate seldom-used files, and remove or repair damaged files.

5.2 Maintaining Free Space

The XENIX system operates best when at least 15% of the space in each file system is free. In any system, the amount of free space depends on the size of the disk that contains the file system and the number of files on that disk. Since all disks have a fixed amount of space, it is important to carefully control the number of files stored on the disk.

If a file system has less than 15% free space, system operation usually becomes sluggish. If no free space is available, the system stops any attempts to write to the file system. This means that the user's normal work on the computer (creating new files and expanding existing ones) stops.

The only remedy for a file system that has less than 15% free space is to delete one or more files from the file system. The following sections describe strategies for keeping the free space available.

5.2.1 Strategies for Maintaining Free Space

The system manager should regularly check the amount of free space of all mounted file systems and remind users to keep their directories free of unused files. You can remind users by including a reminder in the message of the day file */etc/motd*. (See Section 8.11.3 in Chapter 8, "Solving System Problems").

If the amount of free space slips below 15%, the system manager should:

1. Send a system-wide message asking users to remove unused files.
2. Locate exceptionally large directories and files and send mail to the owners asking them to remove unnecessary files.
3. Locate and remove temporary files and files named *core*.
4. Clear the contents of system log files.

Finally, if the system is chronically short of free space, it may be necessary to create and mount an additional file system.

5.2.2 Displaying Free Space

You can find out how much free space exists in a particular file system with the **df** (for “disk free”) command. This command displays the number of *blocks* available on the specific file system. A block is 512 characters (or bytes) of data.

The **df** command has the form:

```
df specialfile
```

where *specialfile* can be the name of a XENIX special file that corresponds to the disk drive that contains the file system (see Appendix A, “XENIX Special Device Files”). If you do not give a special filename, then the free space of all normally mounted file systems is given.

For example, to display the free space of the root file system */dev/root*, type:

```
df /dev/root
```

and press the RETURN key. The command displays the special filename and the number of free blocks. You may compute the percentage of free space by comparing the displayed value with the total number of blocks in the file system. See Appendix A, “XENIX Special Device Files,” for a list of the total blocks.

5.2.3 Sending a System-Wide Message

If free space is low, you may send a message to all users on the system with the **wall** (for “write to all”) command. This command copies the messages you type at your terminal to the terminals of all users currently logged in.

To send a message, type:

```
wall
```

and press the RETURN key. Type the message, pressing the RETURN key to start a new line if necessary. After you have typed the message, press the CTRL-D key sequence. The command displays the message on all terminals in the system. To leave the **wall** command, press the CTRL-D key sequence. This removes the link to other terminals.

5.2.4 Displaying Disk Usage

You can display the number of blocks used within a directory by using the **du** command. This command is useful for finding excessively large directories and files.

The **du** command has the form:

```
du directory
```

The optional *directory* must be the name of a directory in a mounted file system. If you do not give a directory name, the command displays the number of blocks in the current directory.

For example, to display the number of blocks used in the directory */usr/johnd*, type:

```
du /usr/johnd
```

and press the RETURN key. The command displays the name of each file and directory in the */usr/johnd* directory and the number of blocks used.

5.2.5 Displaying Blocks by Owner

You can display a list of users and the number of blocks they own by using the **quot** (for “quota”) command. The command has the form:

```
quot specialfile
```

The *specialfile* must be the name of the special file that corresponds to the disk drive that contains the file system (see Appendix A, “XENIX Special Devices Files”).

For example, to display the owners of files in the file system mounted on the disk drive */dev/fd1*, type:

```
quot /dev/fd1
```

and press the RETURN key. The command displays a list of the users who have files in the file system and the number of blocks in these files.

5.2.6 Mailing a Message to a User

If a particular user has excessively large directories or files, you may send a personal message to the user with the **mail** command.

To begin sending a message through the mail, type:

```
mail login-name
```

and press the RETURN key. The *login-name* must be the login name of the recipient. To send a message, type the message, press the RETURN key, and then press the CTRL-D key sequence. If the message has more than one line, press the RETURN key at the end of each line. The **mail** command copies the message to the user's mailbox, where the user may view it by using the **mail** command. See the *XENIX User's Guide* for details.

5.2.7 Locating Files

Using the **find** command, you may locate files that have a specified name, size, date, owner, and/or last access date. The **find** command is useful for locating seldom-used and excessively large files.

The **find** command has the form:

```
find directory parameters
```

The *directory* must be the name of the first directory you want to search. (The command will also search all directories within that directory.) The *parameters* are special names and values that tell the command what to search for (see *find(C)* in the *XENIX Reference Manual* for complete details). The most useful *parameters* are:

```
-name file
```

```
-atime number
```

```
-print
```

The “-name” parameter causes the command to look for the specified *file*. The “-atime” parameter causes the command to search for files which have not been accessed for the *number* of days. The “-print” parameter causes the command to display the locations of any files it finds.

For example, to locate all files named *core* in the directory */usr*, type:

```
find /usr -name core -print
```

and press the RETURN key. The command displays the locations of all files it finds.

5.2.8 Locating core and /tmp Files

You can locate *core* and */tmp* (temporary) files with the **find** command.

A *core* file contains a copy of a terminated program. The XENIX system sometimes creates such a file when a program causes an error from which it cannot recover. A */tmp* (temporary) file contains data created as an intermediate step during execution of a program. These files may be left behind if a program contained an error or was prematurely stopped by the user. The name of a */tmp* file depends on the program that created it.

In most cases, the user has no use for either *core* or */tmp* files and they can be safely removed.

When you search for *core* or */tmp* files, it is a good idea to search for files which have not been accessed for a reasonable period of time. For example, to find all *core* files in the */usr* directory which have not been accessed for a week, type:

```
find /usr -name core -atime +7 -print
```

and press the RETURN key.

5.2.9 Clearing Log Files

The XENIX system maintains a number of files, called *log files*, that contain information about system usage. When new information is generated, the system automatically appends this information to the end of the corresponding file, preserving the file's previous contents. This means the size of each file grows as new information is appended. Since the log files can rapidly become quite large, it is important to periodically clear the files by deleting their contents.

You can clear a log file by typing:

```
cat </dev/null >filename
```

where *filename* is the full pathname of the log file you wish to clear. A log file normally receives information to be used by one and only one program, so its name usually refers to that program. Similarly, the format of a file depends on the program that uses it.

In some cases, clearing a file affects the subsequent output of the corresponding program. For example, clearing the file */etc/ddate* forces the next backup to be a periodic backup (see Chapter 6, "Backing Up File Systems"). A list of log files appears in Section B.10 in Appendix B, "XENIX Directories."

5.2.10 Expanding the File System

If free space is chronically low, it may be to your advantage to expand the system's storage capacity by creating and mounting a new file system. When you have mounted the file system, you may use this new file system for your work, or to copy user or system directories to it.

A chronic shortage of space usually results from having more users on the system than the current hard disk can reasonably handle, or from having too many directories or files. In either case, creating a new file system allows some of the users and directories to be transferred from the hard disk, freeing a significant amount of space on the existing file system and improving system operation. For details about creating and mounting file systems, see Chapter 4, "Using File Systems."

5.3 File System Integrity

Since file systems are normally stored on hard and floppy disks, occasional loss of data from the file system through accidental damage to the disks is not unusual. Such damage can be caused by conditions such as an improper system shutdown, hardware errors in the disk drives, or a worn out disk.

Such damage usually affects one or two files, making them inaccessible. In very rare cases, the damage causes the entire file system to become inaccessible.

The XENIX system provides a way to restore and repair a file system if it has been damaged. The `fsck` (for "file system check") command checks the consistency of file systems and, if necessary, repairs them. The command does its best to restore the information required to access the files, but it cannot restore the contents of a file once they are lost. The only way to restore lost data is to use backup files. For details about backup disks, see Chapter 6, "Backing Up File Systems."

5.3.1 Repairing the File System

You can repair a file system with the `fsck` command. The command has the form:

```
fsck specialfile
```

The *specialfile* must be the name of the special file that corresponds to the disk drive that contains the file system (see Appendix A, "XENIX Special Device Files").

For example, to check the file system on the disk in the disk drive `/dev/fd1`, type:

`fsck /dev/fd1`

and press the RETURN key. The program checks the file system and reports on its progress with the following messages:

- ** Phase 1 - Check Blocks and Sizes
- ** Phase 2 - Pathnames
- ** Phase 3 - Connectivity
- ** Phase 4 - Reference Counts
- ** Phase 5 - Check Free List

If a damaged file is found during any one of these phases, the command asks if it should be repaired or salvaged. Type `y` (for “yes”) to repair a damaged file. You should always allow the system to repair damaged files, even if you have copies of the files elsewhere or intend to delete the damaged files.

Note that the `fsck` command deletes any file that it considers too damaged to be repaired. If you suspect a file system problem and wish to try to save some of the damaged file or files, check other possible remedies before you invoke the command.

5.3.2 Automatic File System Check

The XENIX system sometimes requests a check of the file system when you first start it. This usually occurs after an improper shutdown (for example, after a power loss). The file system check repairs any files disrupted during the shutdown. For details, see Section 2.2.2 in Chapter 2, “Starting and Stopping the System.”

Chapter 6

Backing Up File Systems

- 6.1 Introduction 6-1
- 6.2 Strategies for Backups 6-1
- 6.3 Using the sysadmin Program 6-2
 - 6.3.1 Creating Backups 6-2
 - 6.3.2 Getting a Backup Listing 6-3
 - 6.3.3 Restoring a Backup File 6-4
- 6.4 Using the tar Command 6-5
 - 6.4.1 Copying files to tar Media 6-5
 - 6.4.2 Restoring Files From tar Media 6-5



6.1 Introduction

A file system backup is a copy of the files in the root directory. A backup allows you to save a copy of the file system as it was at a specific time. You can use the copy later to restore files that are accidentally lost or temporarily removed from the file system to save space.

You can make backups on floppy disk, disk cartridge, or tape cartridge.

This chapter explains how to create backups of the root directory, usr directory, and other file systems, and how to restore files from the backups.

6.2 Strategies for Backups

You should back up the root and usr directories (and any other mounted file systems) on a regular basis. In particular, you should make daily copies of all files modified during the day, and make periodic (e.g., weekly) copies of the entire root and usr directories and other mounted file systems.

The XENIX system offers two ways to back up file systems, the **sysadmin** program and the **tar** command.

The **sysadmin** program is a formal maintenance program for systems that require a rigorous schedule of file system backups. Such systems usually have many users and a large number of files that are modified daily. **Sysadmin** automatically locates modified files, copies them to the backup media, and optionally produces a list of the files. Only a super-user can run **sysadmin**.

A typical backup schedule includes a daily backup once a day and a periodic backup once a week. A daily backup copies only those files modified since the last periodic backup, and a periodic backup copies all files in the file system. The appropriate schedule for a system depends on how heavily the system is used and how often files are modified. In all cases, a periodic backup should be done at least once a month.

The **tar** command is useful on systems with one or two users, or on any system where ordinary users wish to make personal copies of their directories. **Tar** lets the system manager or user choose the files and directories to be copied. However, it does not automatically locate modified files.

The system manager should schedule backups at times when few (if any) users are on the system. This ensures that the most recent version of each file is copied correctly.

You should save daily backup media at least two weeks; periodic backup media should be saved indefinitely. Label the backup media properly with the date of the backup and the names of the files and directories contained in the backup. After a backup has expired, you can use the backup media to create

new backups.

6.3 Using the sysadmin Program

The **sysadmin** program performs daily and periodic backups, lists backup files, and restores individual files from backup media. It presents each task as an item in a menu. To perform a task, simply choose the appropriate item from the menu and supply the required information.

6.3.1 Creating Backups

Before making backups, be sure you have formatted enough backup media. The exact number you'll need depends on the media you are using and the number of files to be copied.

To create a backup, follow these steps:

1. Log in as the super-user.
2. Type:

```
sysadmin
```

and press the RETURN key. The program displays a file system maintenance menu:

```
File System Maintenance
-----
Type 1  to do daily backup (root file system)
      2  to do daily backup (usr file system)
      3  to do periodic backup (root file system)
      4  to do periodic backup (usr file system)
      5  to get a backup listing
      6  to restore file(s)
      7  to do daily backup (other file system)
      8  to do periodic backup (other file system)
      q  to quit
```

3. Choose the appropriate number to do either a daily or periodic backup of the root or usr file systems. You can also make backups of other XENIX file systems. Note that if you have never done a periodic backup, **sysadmin** automatically performs one, even if you chose a daily backup.

If you choose to make a backup of "other file system," **sysadmin** asks you which drive contains the file system, then which partition you wish to back up.

4. **Sysadmin** asks the type of media and the drive number to which you are making backups. Answer the prompts and press the RETURN key.
5. Next, you are prompted to insert the backup media into the appropriate drive. Do so and press the RETURN key.
6. **Sysadmin** displays the current date and the date of the last periodic backup if you are doing a daily backup or "the epoch" if you are doing a periodic backup or if you have never done a periodic backup. The system then begins to copy files to the backup media. If the media runs out of space, **sysadmin** displays the message:

Please insert new volume, then press ENTER:

or

Please change tape, then press ENTER:

7. Remove the current backup media and insert a new one. Press the RETURN key. Continue this way until **sysadmin** displays the message:

DONE

When doing a periodic backup, you may need to repeat the last 2 steps several times before the backup is complete. Be sure to label each of the backup media as you remove it from the drive. For example, label the first "Volume 1," the second "Volume 2," and so on.

6.3.2 Getting a Backup Listing

You can keep a record of the files you have backed up by running the **sysadmin** program and selecting item 5 on the menu. **Sysadmin** copies the names of all the files from the backup media to the temporary file */tmp/backup.list*. The listing is especially convenient if you keep detailed records of the files copied in each backup. The backup listing is available after every daily or periodic backup.

To get the listing, follow these steps:

1. Log in as the super-user.
2. Type:

`sysadmin`

and press the RETURN key. **Sysadmin** displays the maintenance menu.

3. Type 5 and press the RETURN key. **Sysadmin** asks you the type of media and the drive number from which you are getting the backup listing. Answer the prompts and press the RETURN key.
4. **Sysadmin** now prompts you to insert the first of the backup media into the appropriate drive. Do so and press the RETURN key.
5. **Sysadmin** reads the filenames from the backup media and places the names in the list file. If **sysadmin** requires another volume, it prompts for it.

You can produce a printed copy of the backup list, if you have a printer enabled, by typing:

```
lpr /tmp/backup.list
```

and pressing the RETURN key. To save space after printing the file, you should remove it from the */tmp* directory with the **rm** command. You must be running the system in multi-user mode to use **lpr**.

6.3.3 Restoring a Backup File

You can restore files from the backup media by invoking the **sysadmin** program and selecting item 6 from the maintenance menu. You'll need the complete set of backup media containing the latest version of the file you wish to restore. You will also need the "full pathname" of the file you wish to restore. This is the name given for the file in the backup listing.

To restore a file, follow these steps:

1. Log in as the super-user.
2. Type:

```
sysadmin
```

and press the RETURN key. **Sysadmin** displays the file system maintenance menu.

3. Type 6 and press the RETURN key. **Sysadmin** asks you the type of backup media and the drive number from which you are going to restore file(s). Answer these prompts and press the RETURN key.
4. You are then prompted to enter the first volume of the backup media. Do so and press the RETURN key.
5. **Sysadmin** then asks you to enter the "full pathname" for the file you wish to restore. You can enter a pathname to a file or to a directory. If

you specify a directory's pathname, all files and directories below the specified directory are restored.

6. Type the pathname and press the RETURN key.
7. You are asked to insert the volume again. Since the correct volume is in place, simply press RETURN. **Sysadmin** displays the inode number of the files you have specified, then asks for the volume number of the backup media containing the files.
8. Insert the appropriate media when prompted. Type the volume number and press the RETURN key. **Sysadmin** searches for the file. If the file is found, **sysadmin** copies the file to your current directory. If the file is not found, **sysadmin** asks for the next volume. Continue this way until all requested files are restored.

6.4 Using the tar Command

The **tar** command copies specified files and directories to and from backup media. On systems with one or two users, it gives the system manager a direct way to make backup copies of the files modified during a day. On systems with many users, it gives ordinary users a way to make personal copies of their own files and directories.

6.4.1 Copying files to tar Media

You can copy a small number of files or directories to backup media with the **tar** command. The command has the form:

```
tar cvf specialfile files
```

The *specialfile* must be the name of the special file corresponding to a media device (see Appendix A, "XENIX Special Device File"). The drive must contain formatted media. The *files* are the names of the files or directories you wish to copy. If you give a directory name, the command copies all files in the directory (including subdirectories) to the backup media.

For example, to copy the files a, b, and c to a floppy disk in the disk drive */dev/fd0*, type:

```
tar cvf /dev/fd0 a b c
```

and press the RETURN key.

6.4.2 Restoring Files From tar Media

You can also use the **tar** command to restore files from a disk. The command simply copies all files on the disk to your current directory. In this case, the

command has the form:

```
tar xvf specialfile
```

The *specialfile* must be the name of a special file corresponding to the drive containing the tar media.

For example, to restore files from a floppy disk in drive */dev/fd0*, type:

```
tar xvf /dev/fd0
```

and press the RETURN key. Tar copies files on the disk in the specified drive to the current directory.

Since the **tar** command copies files only to the current directory, make sure you are in the desired directory before you invoke the command. You can change to the desired directory with the **cd** command.

Chapter 7

Using Peripheral Devices

7.1 Introduction 7-1

7.2 Adding a Terminal 7-1

7.3 Setting Terminal Lines 7-2

7.3.1 The Gettydefs File 7-2

7.3.2 Changing the Gettydefs File 7-3

7.3.3 Checking the Terminal Settings 7-5

7.4 Changing Serial Line Operation 7-6

7.5 Setting the Terminal Type 7-7

7.6 Removing a Terminal 7-8

7.7 Adding a Lineprinter 7-9

7.7.1 Installing a Printer: lpinit 7-9

7.7.2 Stopping the Print Spooling Daemon: lpsched 7-12

7.7.3 Changing Printer Configuration: lpadmin 7-13

7.7.4 Creating a Default Printer 7-14

7.7.5 Removing Printers 7-14

7.7.6 Moving Requests Between Printers: lpmove 7-15

7.7.7 Accepting and Rejecting Print Requests: accept 7-15

7.7.8 Enabling and Disabling Printers 7-16

7.7.9 Printer Interface Programs 7-16

7.8 Using the Dial Program 7-18

7.8.1 Setting XENIX for Tandy/Radio Shack Modems 7-18

7.8.2 Using a Hayes Modem 7-19

7.8.3 Dial-in vs Dial-out 7-21

7.8.4 Using cu 7-22

7.8.5 Example Dial Source 7-22

7.1 Introduction

One important task of the system manager is to add peripheral devices such as terminals, hard disks, and lineprinters to the system. Adding such devices lets more users use the system, gives extra storage space for users' files and directories, and adds to the system's overall capabilities.

To add a peripheral device, the system manager must make the physical connection between the device and the computer, then use the correct system commands to enable the device for operation. This chapter explains how to use system commands to enable a device for use. It also describes how to maintain the devices once they are added.

Note that all physical connections between a device and the system are device-dependent. For information about these connections, see the hardware manual provided with the device and your computer.

Make sure that any serial wire is connected to your modem, terminal, printer, etc. or not connected to your computer at all. An unterminated wire connected to your computer can considerably reduce system performance. Always disconnect unused lines at your computer, not at the device.

7.2 Adding a Terminal

Extra terminals allow two or more users to use the system simultaneously. Many different terminals work well with the XENIX operating system. A short list of recommended models is given in *terminals(M)* in the XENIX *Reference Manual*.

Before you can add a terminal, you must know how to connect the terminal to a serial line on the computer. You also need to know the name of the serial line. Physical connections for the terminal are usually explained in the terminal's hardware manual. The names of the available serial lines on the system are given in Appendix A, "XENIX Special Device Files." Once a terminal has been connected, you may then enable the terminal for use with the **enable** command.

To add a terminal, follow these steps:

1. Using the recommended procedure in the terminal's hardware manual, connect the terminal to one of the RS-232 serial lines on the computer itself. Make sure that the terminal is compatible with the line configuration.
2. Log in as the super-user.
3. Use the **enable** command to enable the terminal. The command has the form:

enable *specialfile*

where *specialfile* is the name of the serial line to which the terminal is attached. This name depends on your system's configuration (see Appendix A, "XENIX Special Device Files"). For example, the command:

```
enable /dev/tty00
```

enables the terminal connected on serial line */dev/tty00*.

4. Turn on the power to the terminal and press the RETURN key several times. The system should display a "login:" message. When it does, you may log in and begin work.

If no "login:" message appears on the screen, if random characters appear, or if the terminal does not respond to your attempt to log in, you may need to change the baud rate (or line speed) of the terminal to match the serial line. You can change the baud rate with the **stty** command described in the next section.

When using the **enable** command, make sure that you wait a full minute between each use of the command. Failure to do so can cause a system crash.

7.3 Setting Terminal Lines

Your XENIX system can adapt itself to several different terminal baud rates and settings. The same program that prints the login message, **getty**, reads these terminal line values from a table and tries each setting until one is successful. The user can then log in to the system. This table provides several default settings for different kinds of terminals lines. The login process invokes **getty** automatically.

The table of terminal settings is found in a file called */etc/gettydefs*. You can edit *gettydefs* to add different sets of terminal characteristics or change the existing ones.

7.3.1 The Gettydefs File

The file */etc/gettydefs* contains the information that **getty** uses to set up terminal line characteristics such as baud rate. The file is in the form of a table. Each table entry is divided into five fields. These fields are listed in the following form:

```
label# initial-flags #final-flags # login-prompt #next-label
```

<i>label</i>	Identifies the <i>gettydefs</i> entry to getty . This could be a number or a letter. <i>Label</i> corresponds to the line mode
--------------	---

Delete the first line by typing

dd

Delete the third line the same way. Now move the cursor to the last line in the example and press

"1p

The line from the *second* deletion appears:

```
Text contains lines.
Characters form words.
Words form text.
[L]ines contain characters.
~
~
~
~
~
~
~
```

Now type

"2p

The line from the *first* deletion appears:

```
Text contains lines.
Characters form words.
Words form text.
Lines contain characters.
[F]iles contain text.
~
~
~
~
~
~
~
```

Since the text remains in a buffer until it is either pushed off the stack or until you quit the editor, you may use it as many times as you wish.

It is also possible to place text in named buffers. For information on how to create named buffers, see Section 5.3.8.

5.3.17 Searching: / and ?

Vi lets you search forward and backward in a file for patterns. To search forward

1. Press the slash (/) key. The slash appears on the status line.
2. Type the characters you wish to search for.
3. Press RETURN.

If the specified pattern exists, the cursor will move to the first character of the pattern. For example, to search forward in the file for the word "account," type

```
/account
```

Press RETURN. The cursor appears on the first character of the pattern.

To search backward through a file, use ? instead of / to start the search. For example, to find all occurrences of "account" above the cursor, type

```
?account
```

To search for a pattern containing any of the special characters (., * \ [] \$ and ^), you must precede each special character on the status line with backslash. For example, to find the pattern "U.S.A.," type

```
/U\.S\.A\./
```

To continue searching for a pattern, press

```
n
```

after each search. You can continue to use **n** until you specify a new pattern or quit the editor.

Because XENIX is case sensitive, **vi** searches for exactly what you type. To have XENIX disregard case in a search command, you can set the **ignorecase** option before initiating the search:

```
:set ignorecase
```

By default, searches "wrap around" the file. That is, if a search starts in the middle of a file, when **vi** reaches the end of the file it will "wrap around" to the beginning, and continue until it returns to where the search began.

If you do not want searches to wrap around the file, you can change the **wrapscan** option setting. Type

```
:set nowrapscan
```

and press RETURN to prevent searches from wrapping. For more information about setting options, see Section 5.5.

5.3.18 Searching and Replacing

The search and replace commands allow you to perform complex changes to a file in a single command.

The syntax of a search and replace command is

```
g/pattern1/s/[pattern2]/[options]
```

Brackets indicate optional parts of the command line. The *g* tells the computer to execute the replacement on every line in the file. Otherwise the replacement would occur only on the current line. The *options* are explained in the following sections.

To explain these commands we will use the example file from the demonstration:

```
[F]iles contain text.  
Text contains lines.  
Lines contain characters.  
Characters form words.  
Words form text.  
~  
~  
~  
~  
~  
~
```

5.3.18.1 Replacing a Word

To replace the word “contain” with the word “are” throughout the file, type the following command:

```
:g/contain /s//are /g
```

This command says “On each line of the file (*g*), find “contain” and substitute for that word (*s//*) the word “are”, everywhere it occurs on that line (the second *g*)”. Note that a space is included in the search pattern for “contain”;

without the space “contains” would also be replaced.

After the command executes, your screen should look like this:

```
[F]iles are text.
Text contains lines.
Lines are characters.
Characters form words.
Words form text.
-
-
-
-
-
-
```

5.3.18.2 Printing All Replacements

To replace “contain” with “are” throughout the file, then print every line changed, use the **p** option:

```
:g/contain /s//are /gp
```

Press RETURN. After the command executes, each line in which “contain” was replaced by “are” is printed on the lower part of the screen. To remove these lines, redraw the screen by pressing CTRL-L.

5.3.18.3 Confirming Replacements

Sometimes you may not want to replace every instance of a given pattern. The **c** option displays every occurrence of pattern and waits for you to confirm that you want to make the substitution. If you press “y” the substitution takes place; if you press RETURN the next instance of “pattern” is displayed.

To run this command on the example file, type

```
:g/contain/s//are/gc
```

Press RETURN. The first instance of “contain” appears on the status line:

```
Files contain text.
      ^^^^^^^
```

Press “y,” then RETURN. The next occurrence of “contain” appears.

5.3.19 Pattern Matching

Search commands often require, in addition to the characters you want to find, a context in which you want to find them. For example, you may want to locate every occurrence of a word at the beginning of a line. Vi provides several special characters that specify particular contexts.

5.3.19.1 Matching the Beginning of a Line

When a caret (^) is placed at the beginning of a pattern, only patterns found at the beginning of a line are matched. For example, the following search pattern only finds “text” when it occurs as the first word on a line:

```
/^text/
```

To search for a caret that appears as text you must precede it with a backslash (\).

5.3.19.2 Matching the End of a Line

When a dollar sign (\$) is placed at the end of a pattern, only patterns found at the end of a line are matched. For example, the following search pattern only finds “text” when it occurs as the last word on a line:

```
/text$/
```

To search for a dollar sign that appears as text you must precede it with a backslash (\).

5.3.19.3 Matching Any Single Character

When used in a search pattern, the period (.) matches any single character except the newline character. For example, to find all words that end with “ed”, use the following pattern:

```
/.ed /
```

Note the space between the *d* and the backslash.

To search for a period in the text, you must precede it with a backslash (\).

5.3.19.4 Matching a Range of Characters

A set of characters enclosed in square brackets matches any single character in the range designated. For example, the search pattern

```
/[a-z]/
```

finds any lowercase letter. The search pattern

```
/[aA]pple/
```

finds all occurrences of “apple” and “Apple”.

To search for a bracket that appears as text, you must precede it with a backslash (\).

5.3.19.5 Matching Exceptions

A caret (^) at the beginning of *string* matches every character *except* those specified in *string*. For example, the search pattern

```
[^a-z]
```

finds anything but a lowercase letter or a newline.

5.3.19.6 Matching the Special Characters

To place a caret, hyphen or square bracket in a search pattern, precede it with a backslash. To search for a caret, for example, type

```
^/
```

If you need to search for many patterns that contain special characters, you can reset the **magic** option. To do this, type

```
:set nomagic
```

This removes the special meaning from the characters ., \, \$, [and]. You can include them in search and replace commands without a preceding backslash. Note that the special meaning cannot be removed from the special characters asterisk (*) and caret (^); these must always be preceded by a backslash in searches.

To restore **magic**, type

```
:set magic
```

For more information about setting options, see Section 5.5.

5.3.20 Undoing a Command: **u**

Any editing command can be reversed with the **undo(u)** command. **Undo** works on both screen-oriented and line-oriented commands. For example, if you have deleted a line and then decide you wish to keep it, press **u** and the line will reappear.

U undoes only the *last* command. For example, if you make a global search and replace, then delete a few characters with the **x** command, pressing **u** will undo the deletions but not the global search and replace.

5.3.21 Repeating a Command: **.**

To repeat a screen-oriented **vi** command, use the **repeat (.)** command. For example, if you have deleted two words by typing

2dw

you may repeat this command as many times as you wish by pressing the period key (**.**). Cursor movement does not affect the repeat command, so you may repeat a command as many times and in as many places in a file as you wish.

The repeat command repeats only the last **vi** command.

5.3.22 Leaving the Editor

There are several ways to exit the editor and save any changes you may have made to the file. One way is to type

:x

and press RETURN. This command replaces the old copy of the file with the new one you have just edited, quits the editor, and returns you to the XENIX shell. Similarly, if you type

ZZ

the same thing happens, except the old copy file is written out *only* if you have made any changes. Note that the **ZZ** command is *not* preceded by a colon, and is not echoed on the screen.

To leave the editor without saving any changes you have made to the file, type

:q!

The exclamation point tells **vi** to quit unconditionally. If you leave out the exclamation point:

:q

vi will not let you quit. You will see the error message

No write since last change (:quit! overrides)

This message tells you to use **:q!** if you really want to leave the editor without saving your file.

5.3.22.1 Saving a File Without Leaving the Editor

There are many occasions when you must save a file without leaving the editor, such as when starting a new shell, or moving to another file. Before you can perform these tasks you must first save the current file with the **write** command:

:w

You do not need to type the name of the file; **vi** remembers the name you used when you invoked the editor. If you invoked **vi** without a filename, you may name the file by typing

:w filename

where *filename* is the name of the new file.

5.3.23 Editing a Series of Files

Entering and leaving **vi** for each new file takes time, particularly on a heavily used system, or when you are editing large files. If you have many files to edit in one session, you can invoke **vi** with more than one filename, and thus edit more than one file without leaving the editor, as in

vi file1 file2 file3 file4 file5 file6

You can also invoke **vi** using the special characters as abbreviations. For example, to invoke **vi** on the above files without typing each name, type

vi file*

This invokes **vi** on all files that begin with the letters "file." You can plan your filenames to save time in later editing. For example, if you are writing a document that consists of many files, it would be wise to give each file the same filename extension, such as ".s". Then you can invoke **vi** on the entire document:

vi *.s

You can also invoke **vi** on a selected range of files:

vi [3-5]*.s

or

vi [a-h]*

To invoke **vi** on all files that are five letters long, and have any extension:

vi ??????.*

For more information on using special characters, see Section 3.4.4.

When you invoke **vi** with more than one filename, you will see the following message when the first file appears on the screen:

x files to edit

After you have finished editing a file, save it with the **write** command (**:w**), then go to the next file with the **next** command:

:n

The next file appears, ready to edit. It is not necessary to specify a filename; the files are invoked in alphabetical (or numerical, if the filenames begin with numbers) order.

If you forget what files you are editing, type

:args

The list of files appears on the status line. The current file is enclosed in square brackets.

To edit a file out of order, such as *file4* after *file2*, type

:e file4

instead of using the **next** command. If you type

:n

after you finish editing *file4*, you will go back to *file3*.

If you wish to start again from the beginning of the list, type

```
:rew
```

To discard the changes you made and start again at the beginning, type

```
:rew!
```

5.3.24 Editing a New File Without Leaving the Editor

You can start editing another file anywhere on the XENIX system without leaving **vi**. This saves time when you wish to edit several files in one session that are in different directories, or even in the same directory. For example, if you have finished editing */usr/joe/memo* and you wish to edit */usr/mary/letter*, first save the file *memo* with the **write(:w)** command, then type

```
:e /usr/mary/letter
```

/usr/mary/letter appears on your screen just as though you had left **vi**.

Note

You *must* write out your file with the **write (:w)** command if you want to save the changes you have made. If you try to edit a second file without writing out the first file, the message “No write since last change (:e! overrides)” appears. If you use **:e!** all your changes to the first file are discarded.

If you want to switch back and forth between two files, **vi** remembers the name of the last file edited. Using the previous example, if you wish to go back and edit the file */usr/joe/memo* after you have finished with */usr/mary/letter*, type

```
:e#
```

The cursor is positioned in the same location it was when you first saved */usr/joe/memo*.

5.3.25 Leaving the Editor Temporarily: Shell Escapes

You can execute any XENIX command from within **vi** using the shell escape (as in, “escape from **vi**”) command, **!**. For example, to find out the date and time, type

```
!:date
```

The exclamation point sends the remainder of the line to the shell to be executed, and the date and time appear on the **vi** status line. You can use the **!** to perform any XENIX command. To send mail to joe without leaving the editor, type

```
!:mail joe
```

Type your message and send it. (For more information about the XENIX mail system, see Chapter 6, “Mail.”) After you send it, the message

```
[Hit return to continue]
```

appears. Press RETURN to continue editing.

If you want to perform several XENIX commands before returning to the editor, you can invoke a new shell:

```
:sh
```

The XENIX prompt appears. You may execute as many commands as you like. Press CTRL-D to terminate the new shell and return to your file.

If you have not written out your file before a shell escape, you will see the message

```
[No write since last change]
```

It is a good idea to save your file with the **write (:w)** command before executing an escape, just in case something goes wrong. However, once you become an experienced **vi** user, you may wish to turn off this message. To turn off the “No write” message, reset the **warn** option:

```
:set nowarn
```

For more information about setting options in **vi**, see Section 5.5.

5.3.26 Performing a Series of Line-Oriented Commands: Q

If you have several line-oriented commands to perform, you can place yourself temporarily in line-oriented mode by typing

Q

while you are in command mode. A colon prompt appears on the status line.

Commands executed in this mode cannot be undone with the **u** command, nor do they appear on the screen until you re-enter normal **vi** mode. To re-enter normal **vi** mode, type

vi

5.3.27 Finding Out What File You're In

If you forget what file you are editing, press CTRL-G while you are in command mode. A line similar to the following appears on the status line:

```
"memo" [Modified] line 12 of 100 --12%--
```

From left to right, the following information is displayed:

- The name of the file
- Whether or not the file has been modified
- The line number the cursor is on
- The number of lines in the file
- Your location in the file (expressed as a percentage)

This command is also useful when you need to know the line number of the current line for a line-oriented command.

The same information can be obtained by typing

:file

or

:f

5.3.28 Finding Out What Line You're On

To find out what line of the file you are on, type

:nu

and press RETURN. This command displays the current line number and the text of the line.

To display line numbers for the entire file, see Section 5.5.5.

5.4 Solving Common Problems

The following is a list of common problems that you may encounter when using vi, along with the probable solution.

- **I don't know which mode I'm in.**

Press ESC until the bell rings. When the bell rings you are in command mode.

- **I can't get out of a subshell.**

Press CTRL-D to exit any subshell. If you have created more than one subshell (not a good idea, usually), keep pressing CTRL-D until you see the message

[Hit return to continue]

- **I made an inadvertent deletion (or insertion).**

Press “u” to undo the last delete or insert command.

- **There are extra characters on my screen.**

Press CTRL-L to redraw the screen.

- **When I type, nothing happens.**

Vi has crashed and you are now in the shell with your terminal characteristics set incorrectly. To reset the keyboard, slowly type

stty sane

then press CTRL-J or LINEFEED. Pressing CTRL-J instead of RETURN is important here, since it is quite possible that the RETURN key will not work as a newline character. To make sure that other terminal characteristics have not been altered, log off, turn your terminal off, turn your terminal back on, and then log back in. This should guarantee that your terminal's characteristics are back to normal. This procedure may vary somewhat depending on the terminal.

- **The system crashed while I was editing.**

Normally, **vi** will inform you (by sending you mail) that your file has been saved before a crash. The file can be recovered by typing

```
vi -r filename
```

If **vi** was unable to save the file before the crash, it is irretrievably lost.

- **I keep getting a colon on the status line when I press RETURN**

You are in line-oriented command mode. Type

```
vi
```

to return to normal **vi** command mode.

- **I get the error message “Unknown terminal type [Using open mode]” when I invoke vi.**

Your terminal type is not set correctly. To leave open mode, press ESC, then type

```
:wq
```

and press RETURN. Turn to Section 5.5.1 for information on how to set your terminal type correctly.

5.5 Setting Up Your Environment

You can set a number of options to affect your terminal type, how files and error messages are displayed on your screen, and how searches are performed. These options can be set with the **set** command while you are editing, or they can be placed in the **vi** startup file, *.exrc*. (The *.exrc* file is explained in Section 5.5.13.) The following sections describe the most commonly used options and how to set them. There is a complete list of options in **vi(C)** in the *XENIX Reference Manual*.

5.5.1 Setting the Terminal Type

Before you can use **vi**, you must set the terminal type (if this has not already been done for you) by defining the **TERM** variable in your *.profile* file. The **TERM** variable is a string that tells the operating system what type of terminal you are using. To determine this string, you must find out what type of terminal you are using. Then look up this type in *terminals(M)* in the *XENIX Reference Manual*. If you cannot find your terminal type or its number, consult your system administrator.

For these examples, we will suppose that you are using an HP 2621 terminal. For the HP 2621, the **TERM** variable is “2621”. How you define this variable depends on which shell you are using. You can usually determine which shell you are using by examining the prompt character. The Bourne shell prompts with a dollar sign (\$); the C shell prompts with a percent sign (%).

5.5.1.1 Setting the **TERM** variable: The Visual Shell

If you are using the Visual Shell, the terminal type has already been set, and you do not need to change it.

5.5.1.2 Setting the **TERM** variable: The Bourne Shell

To set your terminal type to “2621,” place the following commands in the file *.profile*:

```
TERM=2621
export TERM
```

5.5.1.3 Setting the **TERM** variable: The C Shell

To set your terminal type to “2621” for the C shell, place the following command in the file *.login*:

```
setenv TERM 2621
```

5.5.2 Setting Options: set

The **set** command displays option settings and lets you set options.

5.5.2.1 Listing the Available Options

To get a list of the options available to you and how they are set, type

```
:set all
```

Your display should look similar to this:

noautoindent	open	noslowopen
autoprint	nooptimize	tabstop=8
noautowrite	paragraphs=IPLPPPQPP Libp	taglength=0
nobeautify	noprompt	ttytype=h19
directory=/tmp	noreadonly	term=h19
noerrorbells	redraw	noterse
hardtabs=8	report=5	warn
noignorecase	scroll=4	window=8
nolisp	sections=NHSHH HU	wrapscan
nolist	shell=/bin/sh	wrapmargin=0
magic	shiftwidth=8	nowriteany
nonumber	noshowmatch	

This chapter discusses only the most commonly used options. For information about the options not covered in this chapter, see *vi(C)* in the XENIX *Reference Manual*.

5.5.2.2 Setting an Option

To set an option, use the **set** command. For example, to set the **ignorecase** option so that case is *not* ignored in searches, type

```
:set noignorecase
```

5.5.3 Displaying Tabs and End-of-Line: list

List causes the “hidden” characters and end-of-line to be displayed. The default setting is *nolist*. To display these characters, type

```
:set list
```

Your screen is redrawn. The dollar sign (\$) represents end-of-line and CTRL-I (^I) represents the TAB character.

5.5.4 Ignoring Case in Search Commands: ignorecase

By default, case is significant in search commands. To disregard case in searches, type

```
:set ignorecase
```

To change this option, type

```
:set noignorecase
```

5.5.5 Displaying Line Numbers: **number**

It is often useful to know the line numbers of a file. To display these numbers, type

```
:set number
```

This redraws your screen. Numbers appear to the left of the text. To remove line numbers, type

```
:set nonumber
```

5.5.6 Printing the Number of Lines Changed: **report**

The **report** option tells you the number of lines modified by a line-oriented command. For example,

```
:set report=1
```

reports the number of lines modified, if more than one line is changed. The default setting is

```
report=5
```

which reports the number of lines changed when more than five lines are modified.

5.5.7 Changing the Terminal Type: **term**

If you are logged in on a terminal that is a different type than the one you normally use, you can check the terminal type setting by typing

```
:set term
```

See Section 5.5.1 for more information about TERM variables.

5.5.8 Shortening Error Messages: **terse**

After you become experienced with **vi**, you may want to shorten your error messages. To change from the default (**noterse**), type

```
:set terse
```

As an example of the effect of **terse**, when **terse** is set the message

```
No write since last change, quit! overrides
```

becomes

```
No write
```

5.5.9 Turning Off Warnings: **warn**

After you become experienced with **vi**, you may want to turn off the error message that appears if you have not written out your file before a shell escape (**:!**) command. To turn these messages off, type

```
:set nowarn
```

5.5.10 Permitting Special Characters in Searches: **nomagic**

The **nomagic** option allows the inclusion of the special characters (**. \ \$ []**) in search patterns without a preceding backslash. This option does *not* affect caret (^) or asterisk (*); they must be preceded by a backslash in searches regardless of **magic**. To set **nomagic**, type

```
:set nomagic
```

5.5.11 Limiting Searches: **wrapscan**

By default, searches in **vi** “wrap” around the file until they return to the place they started. To save time you may want to disable this feature. Use the following command:

```
:set nowrapscan
```

When this option is set, forward searches go only to the end of the file, and backward searches stop at the beginning.

5.5.12 Turning on Messages: **mesg**

When you invoke **vi**, write permission to your screen is automatically turned off, preventing **write** messages from appearing. If you wish to receive **write** messages while in **vi**, reset this option as follows:

```
:set mesg
```

To remove the message from your display you must press CTRL-L.

5.5.13 Customizing Your Environment: The .exrc File

Each time **vi** is invoked, it reads commands from the file named *.exrc* in your home directory. This file sets your preferred options so that they do not need to be set each time you invoke **vi**. A sample *.exrc* file follows:

```
set number
set ignorecase
set nowarn
set report=1
```

Each time you invoke **vi** with the above options, your file is displayed with line numbers, case is ignored in searches, warnings before shell escape commands are turned off, and any command that modifies more than one line will display a message indicating how many lines were changed.

5.6 Command Summary

The following tables contain all the basic commands discussed in this chapter.

5.6.1 Entering Vi

Typing this:	Does this:
vi <i>file</i>	Starts at line 1
vi +n <i>file</i>	Starts at line <i>n</i>
vi + <i>file</i>	Starts last line
vi +/pattern <i>file</i>	Starts at <i>pattern</i>
vi -r <i>file</i>	Recovers <i>file</i> after a system crash

5.6.2 Cursor Movement

Pressing this key:	Does this:
h	Moves one space left
l	Moves one space right
SPACEBAR	Moves one space right
w	Moves one word right
b	Moves one word left
k	Moves one line up
j	Moves one line down
RETURN	Moves one line down
H	Moves to the upper-left corner of the screen.
M	Moves to the middle line of the screen.
L	Moves to the bottom line of the screen.
\	Moves to end of sentence.
(	Moves to beginning of sentence.
}	Moves to beginning of paragraph.
;	Moves to end of paragraph.
CTRL-W	Moves to first character of the last word inserted when in insert mode.
CTRL-U	Scrolls up 1/2 screen when in command mode. Moves to the first character of insertion when in insert mode.
CTRL-D	Scrolls down 1/2 screen
CTRL-F	Scrolls down one screen
CTRL-B	Scrolls up one screen

5.6.3 Inserting Text

Pressing	Starts insertion:
----------	-------------------

i	Before the cursor
I	Before first character on the line
a	After the cursor
A	After last character on the line
o	On the following line
O	On the previous line
r	On the current character; replaces one character only
R	On the current character; replaces until ESC

5.6.4 Delete Commands

Command	Function
dw	Deletes a word
d0	Deletes to beginning of line
d\$	Deletes to end of line
ndw	Deletes <i>n</i> words
dd	Deletes the current line
ndd	Deletes <i>n</i> lines

5.6.5 Change Commands

Command	Function
cw	Changes one word
ncw	Changes <i>n</i> words
cc	Changes current line
ncc	Changes <i>n</i> lines

5.6.6 Search Commands

Command	Function	Example
/and	Finds the next occurrence of <i>and</i>	and, stand, grand
?and	Finds the previous occurrence of <i>and</i>	and, stand, grand
/^The	Finds next line that starts with <i>The</i>	The, Then, There
/[bB]ox/	Finds the next occurrence of <i>box</i> or <i>Box</i>	
n	Repeats the most recent search, in the same direction	

5.6.7 Search and Replace Commands

Command	Result	Example
:s/pear/peach/g	All <i>pears</i> become <i>peach</i> on the current line	
:1,\$s/file/directory	Replaces <i>file</i> with <i>directory</i> from line 1 to the end.	<i>filename</i> becomes <i>directoryname</i>
:g/cne/s//1/g	Replaces every occurrence of <i>one</i> with 1.	one becomes 1, one-self becomes 1self, someone becomes some1

5.6.8 Pattern Matching: Special Characters

This character:	Matches:
^	Beginning of a line
\$	End of a line
.	Any single character
[]	A range of characters

5.6.9 Leaving Vi

Command	Result
:w	Writes out the file
:x	Writes out the file, quits vi
:q!	Quits vi without saving changes
!:command	Executes <i>command</i>
:sh	Forks a new shell
!!command	Executes <i>command</i> and places output on current line
:e file	Edits <i>file</i> (save current file with :w first)

5.6.10 Options

This option:	Does this:
all	Lists all options
term	Sets terminal type
ignorecase	Ignores case in searches
list	Displays tab and end-of-line characters
number	Displays line numbers
report	Prints number of lines changed by a line-oriented command
terse	Shortens error messages
warn	Turns off “no write” warning before escape
nomagic	Allows inclusion of special characters in search patterns without a preceding backslash
nowrapscan	Prevents searches from wrapping around the end or beginning of a file
mesg	Permits display of messages sent to your terminal with the write command

